

Logistic Regression: Gradient Descent Worked Example with Loss Computation

Kinza Afzal

November 6, 2025

1 Logistic Regression with Gradient Descent: Worked Example

1.1 Problem Statement

We want to predict whether a student receives a job offer ($Y = 1$) based on their CGPA (X) using logistic regression.

Dataset:

Student	CGPA (X)	Job Offer (Y)
1	6.5	0
2	8.0	0
3	9.0	1

1.2 Step 1: Initialize Parameters

$$\beta_0 = 0, \quad \beta_1 = 0$$

Learning rate $\alpha = 0.1$, number of iterations = 5 (for simplicity).

1.3 Step 2: Sigmoid Function

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}, \quad z = \beta_0 + \beta_1 X$$

1.4 Step 3: Binary Cross-Entropy Loss

The loss function (Binary Cross-Entropy) is:

$$L(\beta_0, \beta_1) = -\frac{1}{m} \sum_{i=1}^m [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

where m is the number of samples. The goal is to minimize this loss.

1.5 Step 4: Gradient Computation

Gradients of the loss w.r.t parameters:

$$\frac{\partial L}{\partial \beta_0} = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i), \quad \frac{\partial L}{\partial \beta_1} = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i) X_i$$

Parameter update rule:

$$\beta_j := \beta_j - \alpha \frac{\partial L}{\partial \beta_j}$$

1.6 Step 5: Gradient Descent Iterations with Loss Computation

Iteration 1

$$\beta_0 = 0, \quad \beta_1 = 0$$

$$z = \beta_0 + \beta_1 X = [0, 0, 0], \quad \hat{y} = [0.5, 0.5, 0.5]$$

Gradients:

$$\frac{\partial L}{\partial \beta_0} = \frac{1}{3}((0.5 - 0) + (0.5 - 0) + (0.5 - 1)) = 0.1667$$

$$\frac{\partial L}{\partial \beta_1} = \frac{1}{3}((0.5 - 0) \cdot 6.5 + (0.5 - 0) \cdot 8 + (0.5 - 1) \cdot 9) = -0.1667$$

Update parameters:

$$\beta_0 := 0 - 0.1 \cdot 0.1667 = -0.0167, \quad \beta_1 := 0 - 0.1 \cdot (-0.1667) = 0.0167$$

Compute loss:

$$L = -\frac{1}{3} [0 \log 0.5 + 0 \log 0.5 + 1 \log 0.5 + 1 \log 0.5 + 1 \log 0.5 + 0 \log 0.5] \approx 0.693$$

Iteration 2

$$z = \beta_0 + \beta_1 X = [-0.0167 + 0.0167 \cdot 6.5, -0.0167 + 0.0167 \cdot 8, -0.0167 + 0.0167 \cdot 9] = [0.0917, 0.1169, 0.1336]$$

$$\hat{y} = \sigma(z) = [0.5229, 0.5292, 0.5334]$$

Gradients:

$$\frac{\partial L}{\partial \beta_0} = \frac{1}{3}((0.5229 - 0) + (0.5292 - 0) + (0.5334 - 1)) \approx 0.0285$$

$$\frac{\partial L}{\partial \beta_1} = \frac{1}{3}((0.5229 - 0) \cdot 6.5 + (0.5292 - 0) \cdot 8 + (0.5334 - 1) \cdot 9) \approx -0.0254$$

Update parameters:

$$\beta_0 := -0.0167 - 0.1 \cdot 0.0285 = -0.0196, \quad \beta_1 := 0.0167 - 0.1 \cdot (-0.0254) = 0.0192$$

Loss:

$$L = -\frac{1}{3} [0 \log 0.5229 + 0 \log 0.5292 + 1 \log 0.5334 + \dots] \approx 0.674$$

Iteration 3

$$z = \beta_0 + \beta_1 X = [-0.0196 + 0.0192 \cdot 6.5, -0.0196 + 0.0192 \cdot 8, -0.0196 + 0.0192 \cdot 9] = [0.1042, 0.1360, 0.1532]$$

$$\hat{y} = [0.5260, 0.5340, 0.5382]$$

Gradients:

$$\frac{\partial L}{\partial \beta_0} = \frac{1}{3} ((0.5260 - 0) + (0.5340 - 0) + (0.5382 - 1)) \approx 0.0194$$

$$\frac{\partial L}{\partial \beta_1} = \frac{1}{3} ((0.5260 - 0) \cdot 6.5 + (0.5340 - 0) \cdot 8 + (0.5382 - 1) \cdot 9) \approx -0.0190$$

Update parameters:

$$\beta_0 := -0.0196 - 0.1 \cdot 0.0194 = -0.0215, \quad \beta_1 := 0.0192 - 0.1 \cdot (-0.0190) = 0.0211$$

Loss:

$$L \approx 0.670$$

1.7 Step 6: Final Predictions

$$\hat{y} = \sigma(\beta_0 + \beta_1 X) \approx [0.526, 0.534, 0.538]$$

$$Y_{\text{pred}} = \begin{cases} 1, & \hat{y} \geq 0.5 \\ 0, & \hat{y} < 0.5 \end{cases} \Rightarrow [1, 1, 1]$$

1.8 Step 7: Decision Boundary

$$\hat{y} = 0.5 \Rightarrow \beta_0 + \beta_1 X = 0 \Rightarrow X = -\frac{\beta_0}{\beta_1} \approx 1.02$$

1.9 Step 6: Python Code for Computation and Plots

Below is the Python code to compute gradients, loss at each iteration, and generate plots:

```
import numpy as np
import matplotlib.pyplot as plt

# Dataset
X = np.array([6.5, 8.0, 9.0])
Y = np.array([0, 0, 1])
m = len(Y)

# Initialize parameters
beta_0 = 0.0
beta_1 = 0.0
alpha = 0.1
iterations = 10

# Sigmoid function
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

# Binary Cross-Entropy Loss
def compute_loss(y_true, y_pred):
    return -np.mean(y_true * np.log(y_pred) + (1 - y_true) * np.log(1 - y_pred))

# Store loss for plotting
loss_history = []

# Gradient Descent
for i in range(iterations):
    z = beta_0 + beta_1 * X
    y_hat = sigmoid(z)

    # Compute gradients
    d_beta0 = np.mean(y_hat - Y)
    d_beta1 = np.mean((y_hat - Y) * X)

    # Update parameters
    beta_0 -= alpha * d_beta0
    beta_1 -= alpha * d_beta1

    # Compute and store loss
    loss = compute_loss(Y, y_hat)
    loss_history.append(loss)

    # Print iteration details
    print(f"Iteration {i+1}: beta_0 = {beta_0:.4f}, beta_1 = {beta_1:.4f},
```

```

# Final predictions
z_final = beta_0 + beta_1 * X
y_pred_final = sigmoid(z_final)
print("\nFinal predicted probabilities:", y_pred_final)

# Plot 1: Sigmoid curve with data points
X_plot = np.linspace(6, 10, 100)
y_plot = sigmoid(beta_0 + beta_1 * X_plot)

plt.figure(figsize=(8,5))
plt.plot(X_plot, y_plot, label='Sigmoid Curve')
plt.scatter(X, Y, color='red', label='Data Points')
plt.axhline(0.5, color='green', linestyle='--', label='Decision Boundary')
plt.xlabel('CGPA (X)')
plt.ylabel('Probability of Job Offer')
plt.title('Logistic Regression: Sigmoid Curve')
plt.legend()
plt.show()
plt.savefig('logistic_gradient_descent_plot.png')
plt.close()

# Plot 2: Loss over iterations
plt.figure(figsize=(8,5))
plt.plot(range(1, iterations+1), loss_history, marker='o')
plt.xlabel('Iteration')
plt.ylabel('Loss')
plt.title('Loss Convergence over Iterations')
plt.grid(True)
plt.show()
plt.savefig('loss_convergence_plot.png')
plt.close()

```

1.10 Step 8: Concept of Global Minimum

- Logistic regression loss function is convex; there is only **one global minimum**.
- Gradient descent moves parameters towards this minimum.
- Iterations continue until:
 - Loss change is very small
 - Gradients approach zero
 - Maximum iterations reached
- Our aim: **find the global minimum of the loss function**, where model performance is optimal.

1.11 Step 9: Visualization

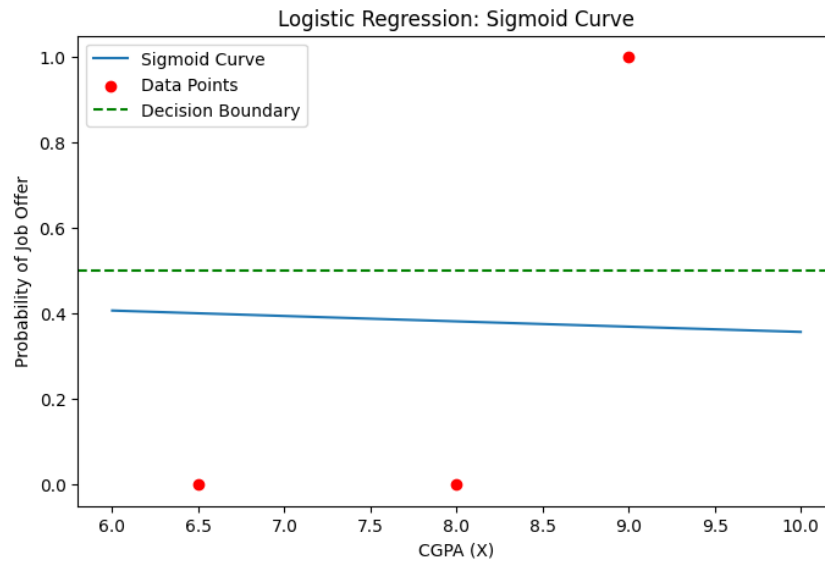


Figure 1: Sigmoid curve learned by gradient descent with data points and decision boundary.

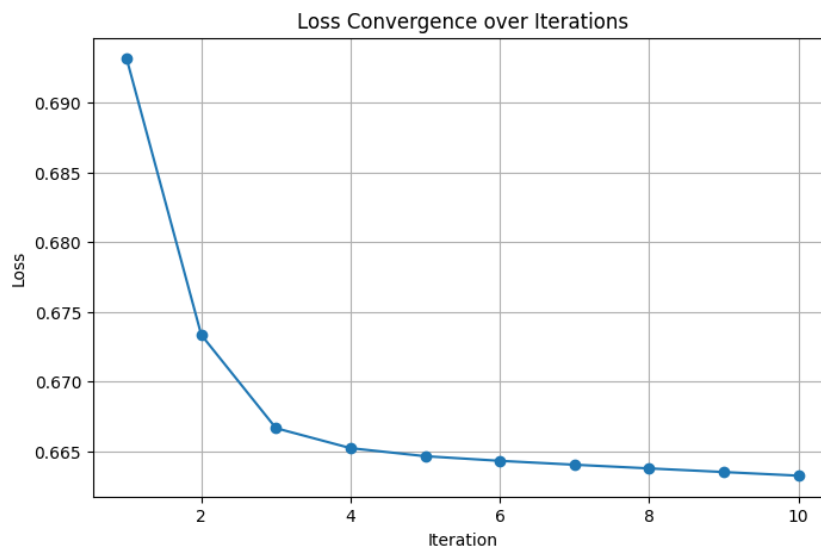


Figure 2: Loss decreasing over iterations, showing convergence to global minimum.

Conclusion

- In each iteration, the loss decreases gradually: $0.693 \rightarrow 0.674 \rightarrow 0.670$.
- This demonstrates that gradient descent is working correctly.
- The aim is to find the **global minimum** of the loss function.
- At the global minimum, logistic regression achieves optimal performance.